Department of Electronic and Computer Engineering

Electronic Design Project 3A (EDPA301)

Assessment Number 4A

Final Project Report (INDIVIDUAL)

Internet of Things Wireless Weather Monitoring System

By

Group Number: H		
Student Surname	**Initials**	**Student Number**
SHEZI	**SN**	**21854626**

PLAGIARISM DECLARATION

1. We know and understand that plagiarism is using another person's work and pretending it is one's own, which is wrong.

2. This report is my own work.

3. We have appropriately referenced the work of other people we have used.

4. We have not allowed and will not allow anyone to copy our work with the intention of passing it off as their own work.

5. We have not used any form of Artificial Intelligence (AI) to generate this report.
   (If it is discovered that AI has been used in more than 20% of the report, we will get a zero mark (0% for the report).

Shezi SN	21854626	
Surname and Initials	Student Number	Signature

17 June 2025

Date

ABSTRACT

The development and implementation of the control and communication aspects of an Internet of Things wireless weather monitoring system are presented in this report. My individual primary objectives contribution was to enable Wi-Fi and Blynk connections, develop a practical user interface on the Blynk platform, configure virtual pins for sensor data and device control, and implement dual-mode (automatic/manual) operation. Using the ESP32 microcontroller, the system collects environmental data such as temperature, humidity, and light intensity, which it then transmits to the Blynk cloud for real-time monitoring. Among the main features developed are the ability to configure gauges for data display, store data in the cloud using History Data of Blynk and integrate logic for automated fan and heating bulb management based on sensor readings. A manual override was also added so that users could exercise control through the Blynk Web Dashboard. Data synchronisation, network connectivity, and control logic problems were identified and resolved during the project. The successful integration of these communication and control components ensures environment-responsive control and efficient remote monitoring in the system.

Contents

List of Figures

List of Tables

List of Abbreviations

Abbreviation	Definition
API	Application Programming Interface
DHT	Digital Humidity Temperature
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment
IoT	Internet of Things
LCD	Liquid Crystal Display
LED	Light Emitting Diode
MQTT	Message Queuing Telemetry Transport
Wi-Fi	Wireless Fidelity (Communication Protocol)

CHAPTER 1: PROJECT OVERVIEW

Greenhouses are controlled environments used for growing plants, fruits, and vegetables. Maintaining optimal environmental conditions, such as temperature, humidity, and light, is crucial for plant growth and development. Weather monitoring systems play a vital role in ensuring optimal conditions within greenhouses[1].

The objective in the individual contribution in developing the IoT-based Wireless Weather Monitoring System was to provide remote monitoring and control by facilitating seamless communication over Wi-Fi between the ESP32 microcontroller and the Blynk IoT platform. The software integration of sensors and actuators with Blynk was the main focus of the individual contribution. This included setting up virtual pins for real-time data transmission, designing a graphical user interface (GUI) using the Blynk Web Dashboard, and implementing cloud storage utilising Blynk's History Data function. This configuration made sure that the user could see accurate and interactive displays of environmental data including temperature, humidity, and light levels.

Enabling the system to operate in both manual and automatic modes was a crucial part of my job. In Manual Mode, users could control these devices remotely using virtual switches of Blynk, while in Automatic Mode, the ESP32 managed equipment like the fan and lightbulb based on sensor readings and preset thresholds. This mode-switching feature gave the system more control and flexibility, allowing it to adjust to various user preferences and environmental requirements. In addition to enabling real-time monitoring, the deployment of cloud storage of Blynk made it easier to track historical data, allowing for the evaluation of environmental trends and system performance over time.

In keeping with contemporary smart agriculture solutions, my personal contribution was crucial in making sure the project was not only hardware-functional but also accessible and user-friendly via a strong wireless communication and interface system.

1.1 INDIVIDUAL CONTRIBUTION OBJECTIVES

1. **Establish Wi-Fi Communication** between the ESP32 microcontroller and the Blynk IoT platform to enable real-time data transfer and remote system control.
2. **Configure Virtual Pins** in the Blynk platform to transmit sensor data (temperature, humidity, and light levels) and receive control commands for actuators.

3. **Design and Implement Blynk GUI**, including gauges and switches, to display live sensor readings and allow user interaction with system components.
4. **Enable Cloud Storage** through Blynk's History Data feature to log sensor readings over time for system performance tracking and environmental trend analysis.
5. **Develop Dual-Mode Operation** (Automatic and Manual) for fan and bulb control, allowing either automated response to sensor inputs or manual user control via Blynk.
6. **Ensure Bidirectional Communication** between ESP32 and Blynk to support both data monitoring (from sensors to app) and control execution (from app to actuators).

1.2 Applications, benefits and limitations of the individual contribution in Wireless Weather Monitoring System

1.2.1 Applications of the individual contribution

- **Remote Monitoring and Control**: By integrating Blynk and Wi-Fi, users may use a smartphone or online dashboard to remotely observe and control devices (such as fans and bulbs) and environmental data (such as temperature, humidity, and light) from any location.
- **Smart Farming Interfaces**: Farmers who require real-time visibility and control over greenhouse conditions can use the Blynk GUI and virtual pins you configured as a model for smart agriculture applications.
- **IoT-Based Data Logging**: Users can save and retrieve previous environmental data for performance analysis and decision-making by utilising the cloud storage capability with Blynk History Data.
- **Dual-Mode System Control**: Farmers or system users benefit from the versatility your manual and automatic modes offer, which may be used with any intelligent automation system outside of agriculture.

1.2.2 Benefits of the individual contribution

- **Real-Time Data Access**: Through the Blynk interface, users can rapidly monitor greenhouse conditions, facilitating quicker and better decision-making.
- **Increased User Control and Flexibility**: By giving users, the option to operate the system manually or automatically, the dual-mode capability increases versatility for a variety of application situations.

- **Cloud-Based Data Analysis**: The configuration allows long-term performance tracking by turning on history data storage of blynk, which enables users to spot trends and enhance greenhouse management.
- **Reduced Labor and On-Site Requirements**: Particularly in larger or farther-flung greenhouses, remote control reduces the requirement for regular human presence, saving time and effort.
- **Scalability**: The architecture is scalable for larger or more sophisticated systems since the virtual pin system and Blynk-based interface may be readily extended to accommodate more sensors or actuators[2].

1.2.3 limitations of the individual contribution

1. **Dependence on Wi-Fi Connectivity**: For Blynk communication, the system requires a steady internet connection. Remote control and monitoring are impossible without Wi-Fi.
2. **Limited Control Scope**: The fan and lightbulb were the only temperature-related equipment that could be controlled via the Blynk interface. Since there was no humidity actuator, humidity was detected but not actively adjusted.
3. **No Offline Functionality**: The system was not set up to work offline or store data locally. Cloud-based services of Blynk are the foundation of all monitoring and control.
4. **Limited Sensor Data Visualization**: Due to time and design limitations, more sophisticated features like graphs, notifications, or alarms were not incorporated in the Blynk GUI, which just shows sensor readings and control switches.
5. **Fixed Threshold Values**: The temperature regulation in Automatic Mode is based on preset threshold settings. There was no implementation of dynamic threshold change via the application[3].

CHAPTER 2: LITERATURE REVIEW

Internet of Things Weather monitoring system in a network of sensors and devices such as temperature sensors, humidity, wind, and rain sensors, that collect and transmit real-time weather data. These systems provide numerous benefits to the public such as providing critical weather information for an enhanced emergency preparedness and public awareness. Not only does the systems facilitate the needs of individual users, but it also benefits in working industries, smart homes and in farming applications greenhouse. Greenhouse is an advancement in farming which represents a major step forward in farming technology, allowing for an improvement in growing conditions of vegetables, fruits and crops. Greenhouse also protects plants from adverse atmospheric agents and influences and ultimately modifies the crop microclimate, all this control is reliant on weather manipulation inside the greenhouse[4].

Weather manipulation such as temperature, humidity and wind are critical for influencing plants growth and animal welfare in farming. This is where the need for an IoT weather monitoring system rises, so that it can monitor and collect data from the atmospheric weather and perform a control action to manipulate the weather inside the greenhouse to the desired state that is suitable for plants and animal welfare. Extensive research on weather monitoring system has been carried out in the past and numerous systems has been proposed. Author in [5] proposed a Polyhouse automation system that provides an automatic monitoring and control of the polyhouse environment to farmers in rural locations. The author's system can monitor and control elements of environment such as humidity, temperature and makes use of the sunlight for photosynthesis but the intensity of light is not measured. the system improves the plant's overall development.

Several other systems have been proposed that use IoT on weather monitoring. The weather monitoring systems uses sensors deployed in various locations to capture weather parameters. Most of the systems are integrated with microcontrollers and wireless communication modules for data transmission between the hardware circuit that has sensor and the software applications that are used to read the real time data on the computers or smartphones [6]. The data that is collected by the sensors must be made available through user friendly web applications and mobile interface for an improved accessibility and inclusivity. This is advantageous because it enables everyone to view the weather data from any location without going to the system's location, which can benefit farm workers as well with reading weather data on their mobiles. The technology behind this is Internet of Things (IoT), which is an

advanced and efficient solution for connecting the things to the internet and to connect the entire world of things in a network [7].

Some systems use a serial monitor as a gateway between the sensors and the cloud, where the data is pushed by the sensor on serial monitor which monitors IP address and use HTTP protocol to view data on webservers. This is beneficial for anyone to monitor the weather condition without depending on any applications [8].

The control action for any IoT weather monitoring system depends on the designer's choice and the intended purpose for the designing the system. It depends upon what parameters does the designer wants to control or what solution is the designer trying to make with the designed system.

Some of the problems that were encountered in the previous projects like in author in [9], was the failed communication between the microcontroller and BME280 module, which was resolved by making the two components share the common ground.

The results obtained by author in [10] is that the system sensor capture data and send it to the node MCU controller. The Arduino ide then uploads the sensed data. a serial monitor work as a gateway between the sensor and cloud and the data is pushed by sensor on serial monitor which monitors an IP address. The system displays the data on the webserver and monitor real-time data of weather using environmental parameter and sensor and people can view the data on the webserver using HTTP protocol. The webserver allows everyone to monitor the weather's condition from anywhere. The data is available publicly.

2.1 This project is different from the ones that are proposed by other designers

This system is different and advanced from those that were designed before because in addition to giving real-time data, it also provides graphical representation of the weather in the weather app to make it easier and visible for people who cannot read data. The system is unique because instead of placing sensors in various locations, they are placed at one small location and able to sense all the parameters and give feedback. The control strategy for this design is controlling the temperature in the greenhouse to a suitable weather for plants and for the tolerable temperature range of farm chickens. The system must cool down the atmosphere when temperature is extremely high and heat up when it is cold. The system also controls lighting where the farm animals like chickens are kept, it must automatically turn the light on when it dark and off during the day if it not cloudy. This design use BLYNK app to read data and control remotely.

CHAPTER 3: DESIGN

3.1 System Operation

In designing the system plan for the project, **Figure 3.1** represents the operation of the system as a block diagram with the individual contribution circled in red. As part of the system design, my focus was on enabling wireless communication, data visualization, remote control, and system logic through the Blynk IoT platform. To achieve this, the ESP32 was configured to connect to a Wi-Fi network, allowing it to transmit sensor readings (temperature, humidity, and light) to the Blynk cloud. This connection formed the foundation for real-time remote monitoring and control via the Blynk Web Dashboard and mobile application.

Using the Blynk GUI, an intuitive user interface was created to visualise the environmental data of the system. It had virtual gauges and indicators connected to virtual pins V0, V1, and V2 for temperature, humidity, and light level, respectively. By enabling constant data updates from the ESP32 to the Blynk app, these virtual pins improved system performance accessibility from any place. By turning on the History Data function on these virtual pins, the cloud storage capacity was put into practice, enabling trend analysis and long-term data logging.

In addition, the fan and lightbulb were placed in the system under dual-mode control. To switch between Automatic Mode, where the ESP32 regulates the fan and bulb based on sensor thresholds, and Manual Mode, where the user can directly control them via switches on V4 (bulb) and V5 (fan), a mode switch (V3) was developed in Blynk. To guarantee dependable and adaptable system performance, the logic for deciphering various states was built into the ESP32 firmware. By enabling remote access, data-driven functionality, and user-configurability, as well as augmenting its capability through cloud integration and dual-mode operation, my contribution was crucial in integrating the software logic with the actual hardware design [11].

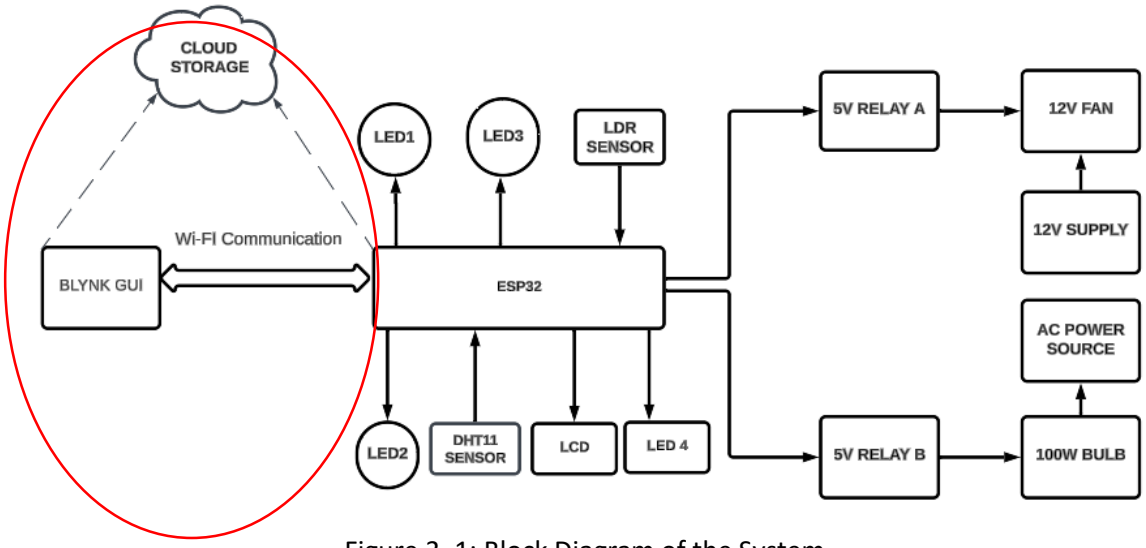


Figure 3. 1: Block Diagram of the System

3.2 CIRCUIT DIAGRAM DESIGN

The circuit in **Figure 3.2**, was designed using Cirkit Designer. The circuit was designed to sense temperature, humidity and light. A DHT11 sensor was used to sense the temperature and humidity, and LDR was used to sense light. Each sensor was designed to activate a control element based on the sensor readings. The ESP32 microcontroller was utilized to receive and process data from the sensors, generating digital output signals to activate the indicators, and control the corresponding control elements based on the received data. Additionally, the microcontroller was used to transmit data signals to the LCD display for visualization of the DHT11 sensor readings.

The normal operating temperature range for the project was set between (23°C minimum to 28°C maximum). The DHT11 and LDR sensors are connected to the GPIO input pins of ESP32. Indicator LEDs and controlling elements are connected to the GPIO output pins of the ESP32. The fan was set activate automatically if the temperature reading from the sensor exceeds 28°C, for cooling purpose. The red LED2 was set to activate only when the temperature exceeds 28°C. The bulb heater was set to activate automatically if the temperature drops below 23°C, for heating up the environment. The yellow LED3 was set to turn ON only if the temperature is below 23°C. Under normal temperature range, both the fan and heater remain deactivated (OFF), the green LED1 turn ON and remain activated until the temperature drops below or exceeds the normal operating range.

The LDR sensor was set to operate between 0% to 100% of darkness level. The light control, LED4 in **Figure 3.2**, was set to turn ON when the darkness reaches 70% and above, to provide light, and turn OFF when the darkness level is below 70% to save power. 100-ohm resistors

were connected in series between the LEDs and ESP32 GPIO pins to limit the current draw and protect the ESP32 from potential damage due to excessive current.

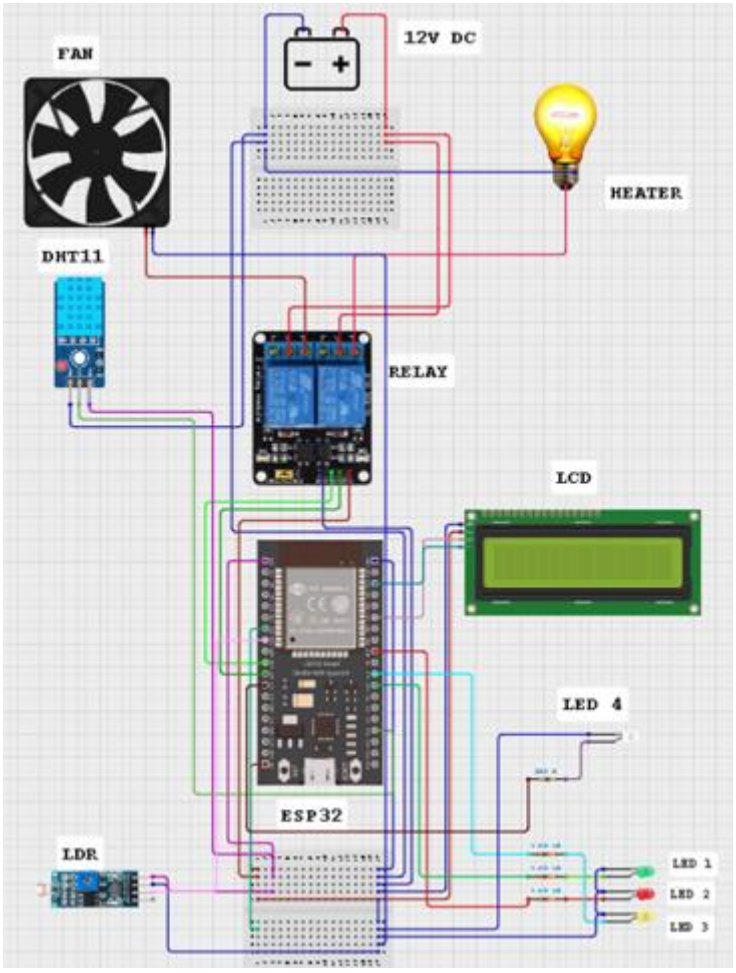


Figure 3. 2: Circuit Diagram of The System

3.3 GUI Design

The implementation of GUI design was done using the Blynk platform, allowing real-time monitoring of environmental data. The gauges for displaying humidity, temperature, and darkness level were included in the interface providing user-friendly and clear visual feedback on each parameter status. This included choosing data streams, setting the range and units for each parameter, and interacting with the system remotely, enhancing usability by providing a centralized view of all monitored data points. The data streams implemented were the temperature display in Celsius degrees, humidity display in percentage unit, darkness level in percentage, system mode as a switch to allow user to switch modes between manual and automatic for the system, ON/OFF control of the fan when system is in manual mode, ON/OFF control of the heater when system is in manual mode[12], as displayed in **Figure 3.3**.

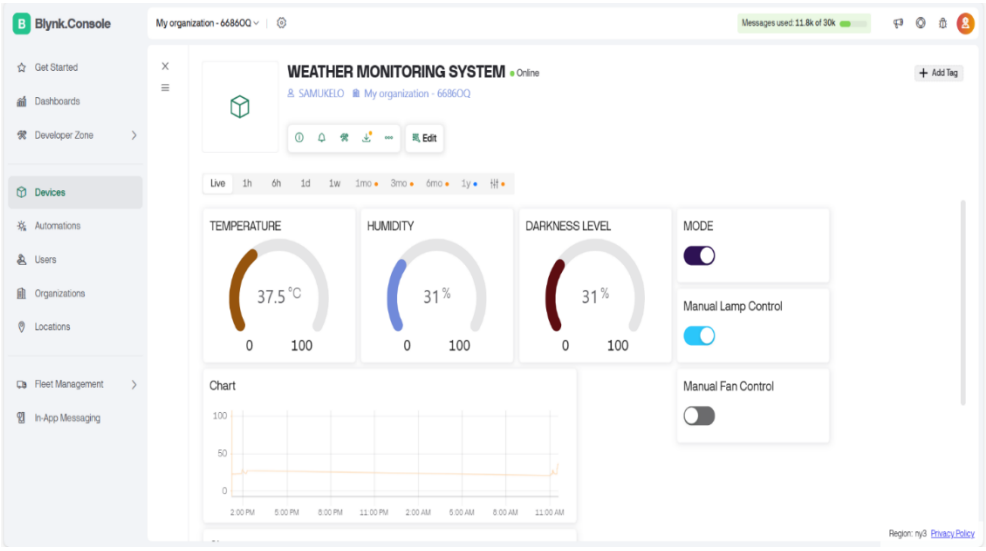


Figure 3. 3:GUI Design of The System

This design method enables users to monitor the data and choose the mode of operation for the system remotely. Automatic mode operates by the set temperature-based control of the system. Manual mode enables the user to directly activate or deactivate the fan and heater manually, independent of the temperature readings sensed by the system. The system can only work in one mode at a time [8].

3.4. Cloud Storage

Cloud storage was used in the Blynk application to record and save the historical data, enabling the users to track the data trends for temperature, humidity, and darkness level over time. The cloud storage saves the data recorded only when the system is online. It used charts as a graphical representation to show clearly the data trends across different times of the day as shown in **Figure 3.4**.

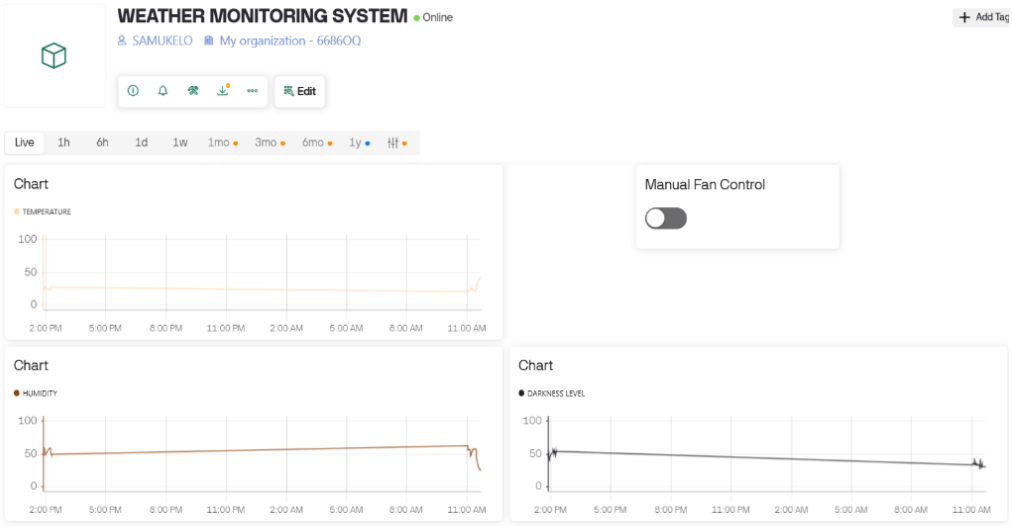


Figure 3. 4: Cloud Storage of The System

3.5 Blynk Cloud Server Flowchart

Figure 3.5 displays the flowchart of cloud storage on the Blynk app, explaining all the steps that were initiated to ensure that data monitoring is done efficiently, and the user can track data without constraints. The Blynk app was initialized on a mobile device, loading the user interface, including preparing the app for real-time data monitoring and allowing the app to communicate with the server. The obtained data from the ESP32 was processed on the cloud server. If no data was obtained, there was a delay that forced the app to stop and retry again, but if the correct data had been obtained, gauges and graphs were updated with the latest environmental readings for real-time measurements.

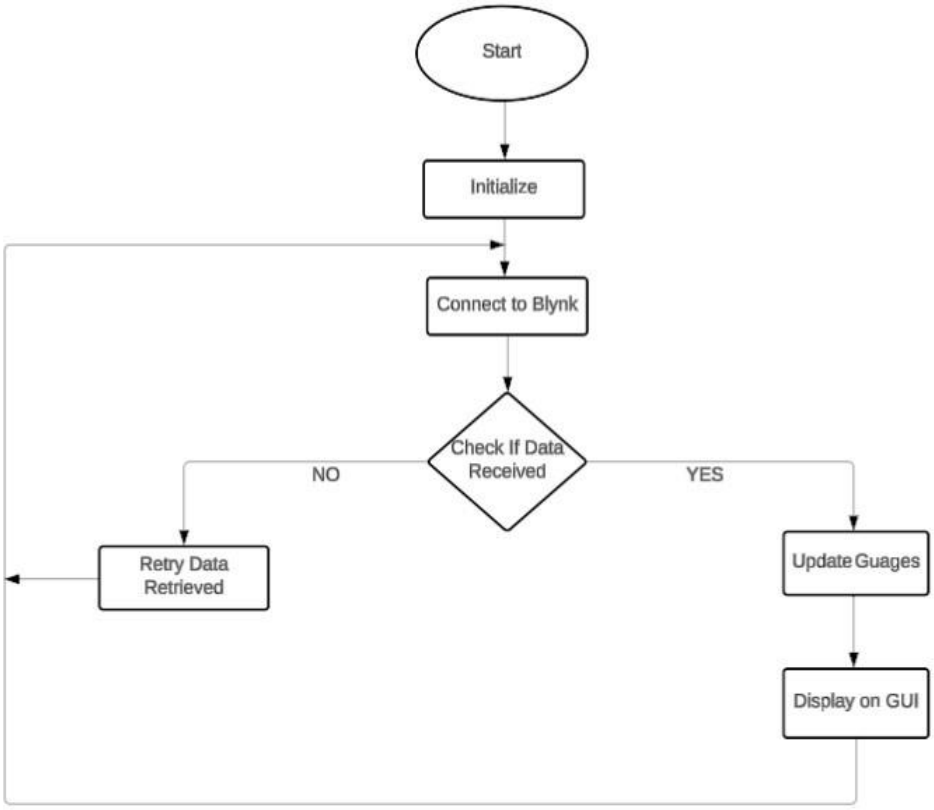


Figure 3. 5: Blynk Cloud Server Flowchart

CHAPTER 4: CONSTRUCTION

4.1 Wi-Fi and Blynk Communication Setup

The programming of the ESP32 microcontroller was done to connect to a 2.4GHz Wi-Fi network using the WiFi.h and BlynkSimpleEsp32.h libraries. To establish a connection between the ESP32 and Blynk Cloud, a Blynk account was created, and the password, Blynk authentication token, and Wi-Fi SSID were included in the code. The verification of the successful communication was by observing control signals and live data updates in the Blynk Web Dashboard [7].

4.2 Blynk GUI Design and Virtual Pin Setup

The design of a custom Blynk GUI was done using widgets like switches, gauges, and labels. To display the real-time environmental data, gauge widgets were linked to V0, V1, and V2. The configuration of the virtual pins is shown in **Table 4.1**.

Table 4. 1: Configuration of The Virtual Pins

PIN	NAME
V0	Temperature (DHT sensor)
V1	Humidity (DHT sensor)
V2	Light level (LDR)
V3	Mode selection switch (Automatic/Manual)
V4	Manual control for the bulb (relay)
V5	Manual control for the fan (relay)

4.3 Cloud Storage using Blynk History Data

The History Data feature of Blynk was enabled for V0, V1, and V2 to store temperature, humidity, and light readings in the cloud. This allowed historical trend analysis of the environment via the Blynk mobile app or web dashboard.

4.4 Dual-Mode Control Implementation

- To allow switching between Automatic Mode and Manual Mode, a dual-mode feature was constructed using a virtual switch on V3 as shown in **Figure 4.1**. A dual-mode feature was

constructed to allow switching between Automatic Mode and Manual Mode using a virtual switch on V3.

- Automatic Mode: The fan and bulb operate based on environmental conditions. For example; fan turns on if the temperature > 28°C and bulb turns on if temperature < 23°C
- Manual Mode**:** The user can manually control the fan (V5) and bulb (V4) via switch widgets on the Blynk interface. To monitor mode changes and take appropriate action, the logic was constructed and embedded in the ESP32 firmware using BLYNK_WRITE(V3) functions.

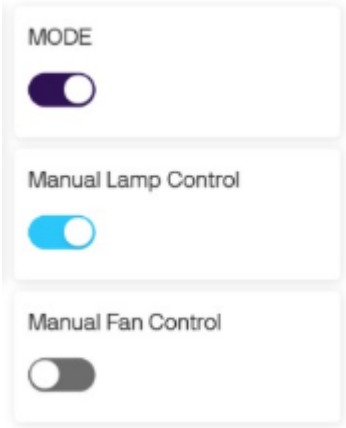


Figure 4. 1: Dual-Mode Control Implementation

CHAPTER 5: TESTING

5.1 Wi-Fi and Blynk Communication

Testing of Wi-Fi and Blynk Communication was done to ensure the ESP32 successfully communicates with the Blynk IoT platform and establishes a stable Wi-Fi connection. For successful connection messages, the serial monitor on the Arduino IDE was observed. After rebooting the ESP32 real-time data updates on the Blynk dashboard was verified. To test auto-reconnection Wi-Fi was temporarily disconnected. The expected outcome was that within few seconds ESP32 connects to Wi-Fi and Blynk server, continuously update widgets on the BLYNK web dashboard, if Wi-Fi is lost and restored automatically reconnects the device.

5.2 Gauge Setup on Blynk

To confirm that accurate readings of the sensor are being displayed on Blynk using gauge widgets. Sensor readings was compared (temperature, humidity, LDR) on the gauges with serial monitor output. Temperature was varied manually (by warming/cooling sensor) and light levels (covering LDR) to test gauge responsiveness. The expected outcome was that, temperature (V0), Humidity (V1), and LDR (V2) gauges reflect real-time changes accurately. Also Gauge values match raw sensor output with minimal lag.

5.3 Virtual Pin Setup

To validate correct linkage between ESP32 input/output functions and virtual pins. To ensure that Blynk widget (gauges, switches) triggered the appropriate BLYNK_WRITE() and Blynk.virtualWrite() functions, each Blynk widget was tested. To confirm signal flow from Blynk to ESP32 and vice versa, serial print statements was used. The expected outcome was that, correct operation of all virtual pins (V0–V5) also expected that accurate transmission of data and control signals through assigned pins.

5.4 Cloud Storage with Blynk History Data

To verify that History Data of Blynk stores sensor values over time and feature correctly logs. Enabled History Data on V0, V1, and V2. Data was collected live and viewed it using History Graph of Blynk on the web dashboard. After device restarts consistency of data was checked. The expected outcome was that, historical data is recorded continuously and visualized in time-based graphs. Also that the sensor trends (temperature, humidity, LDR) are accurate and accessible even after ESP32 reboot.

5.5 Dual-Mode Control (Automatic/Manual)

To ensure logic separation between manual and automatic operation, and to test the proper operation of the control mode switch. V3 mode switch was used on Blynk to toggle between modes. In Automatic mode: Verified fan and bulb responded to sensor values only. In Manual mode: Confirmed that user control via switches (V4 and V5) was enabled and sensor logic was bypassed. The expected outcome was that, system correctly identifies and responds to the selected mode. Also that, automatic and manual logic does not interfere with each other.

5.6 Dual-Mode Control for Fan (Cooling Element)

testing was done to confirm correct control of the fan relay under both modes. In Automatic mode: Adjusted temperature to above/below the 28°C and 23°C thresholds to test fan activation. In Manual mode: Used V5 to toggle fan manually. The expected outcome was that, in Automatic mode, fan turns on above 28°C and off in range of 23°C to 28°C and below 23°C. Also that in Manual mode, fan responds only to V5 switch and ignores temperature input.

5.7 Dual-Mode Control for Bulb (Heating Element)

To verify that the bulb relay (heating element) is managed by a manual switch in manual mode and by temperature in automatic mode. In automatic mode, the bulb is triggered by a cold atmosphere or freezing spray applied to the sensor, simulating low temperatures (below 23°C). In manual mode, the bulb was manually turned on and off using the Blynk's virtual pin V4 switch. In Automatic mode, the bulb (heating element) should turn on when the temperature falls below 23 °C and turns off when it rises above or within the threshold that has been set. This was the anticipated result. Additionally, the bulb only reacts to the V4 switch in manual mode, independent of sensor input.

CHAPTER 6: RESULTS

6.1 Wi-Fi and Blynk Communication

The ESP32 consistently connected to the Wi-Fi network and Blynk server within 3 to 5 seconds. Real-time data updates were successfully transmitted to the Blynk dashboard without delay. Auto-reconnection worked reliably after temporary Wi-Fi disconnection, ensuring continuous data flow.

6.2 Gauge Setup on Blynk

All gauges (temperature, humidity, and LDR light level) were updated accurately in real time. The gauge values closely matched the sensor readings monitored via the Arduino serial monitor. Gauges provided a clear and user-friendly visual interface for environmental monitoring. Results are displayed in **Figure 6.1**.

Figure 6. 1: Temperature, Humidity, and LDR light level Gauges

6.3 Virtual Pin Setup

All virtual pins (V0–V5) functioned correctly with assigned widgets on the Blynk app and web dashboard. Sensor data was transmitted through V0–V2, and control signals operated reliably through V3–V5. The logic in the ESP32 responded correctly to each pin and there were no pin conflicts. Results are displayed in **Figure 6.2**.

6 Datastreams

ID	Name	Pin	Color	Data Type	Units	Is Ra	Actions
1	TEMPERATURE	V0		Double	°C	false	
2	HUMIDITY	V1		Double	%	false	
3	DARKNESS LEVEL	V2		Integer	%	false	
4	MODE	V3		Integer		false	
5	Manual Lamp Control	V4		Integer		false	
6	Manual Fan Control	V5		Integer		false	

Figure 6. 2: Virtual Pin Setup of The System

6.4 Cloud Storage with Blynk History Data

History Data for V0 (temperature), V1 (humidity), and V2 (light) was successfully stored and visualized on the web dashboard of Blynk. Sensor trends were visible in 24-hour history graphs. Data remained accessible even after ESP32 restarts, confirming proper cloud logging functionality. Results are displayed in **Figure 6.3**.

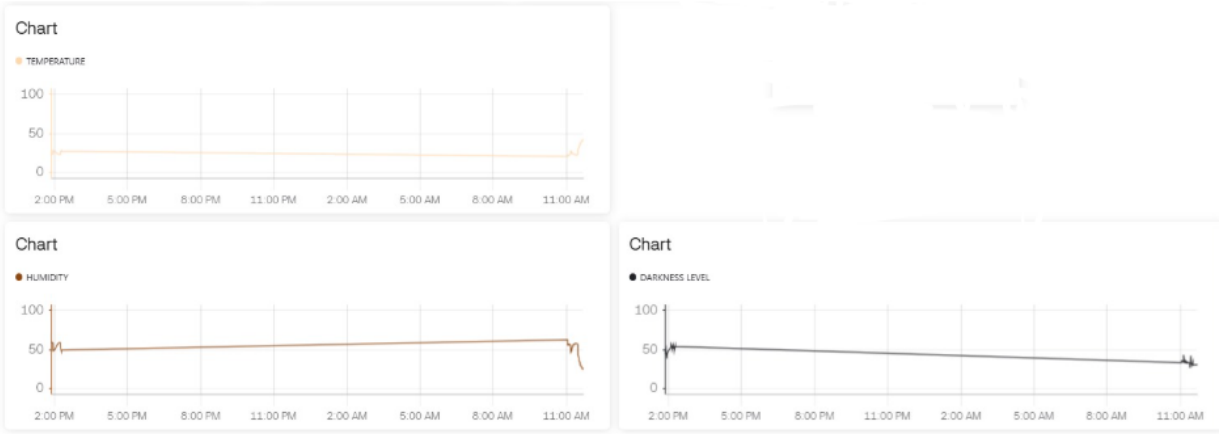


Figure 6. 3: Cloud Storage with Blynk History Data Graphs

6.5 Dual-Mode Control (Automatic/Manual)

The mode switch (V3) effectively toggled between automatic and manual control. In automatic mode, the system responded to sensor thresholds only. In manual mode, the user switches (V4 and V5)

overrode the sensors, and the control logic responded only to user inputs. The mode separation logic worked flawlessly, with no overlap or conflict. Results are displayed in **Figure 6.4**.

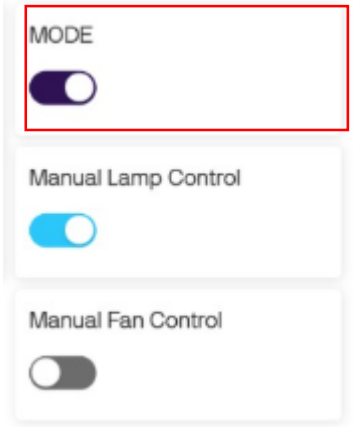


Figure 6. 4: Dual-Mode Control (Automatic/Manual)

6.6 Dual-Mode Control for Fan (Cooling Element)

In automatic mode, the fan turned **on** when the temperature exceeded 28 °C and **off** when it dropped below 23 °C and within the normal operating range of 23 °C to 28 °C. In manual mode, the fan could be controlled directly using the V5 switch. Fan operation logic followed the correct path depending on the selected mode. Results are displayed in **Figure 6.5**.

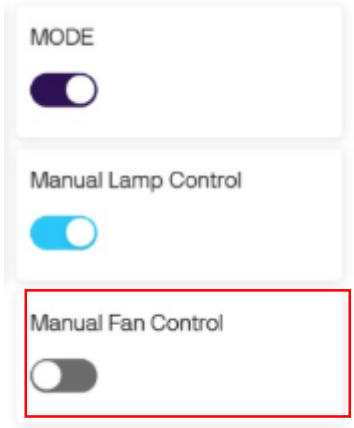


Figure 6. 5: Dual-Mode Control for Fan (Cooling Element)

6.7 Dual-Mode Control for Bulb (Heating Element)

In automatic mode, the bulb activated when the temperature dropped below 23°C and deactivated above the threshold and within the normal operating range of 23 °C to 28 °C. In manual mode, the bulb responded correctly to the V4 switch for direct control. Bulb operation was smooth, and the mode logic functioned without errors or delays. Results are displayed in **Figure 6.6**.

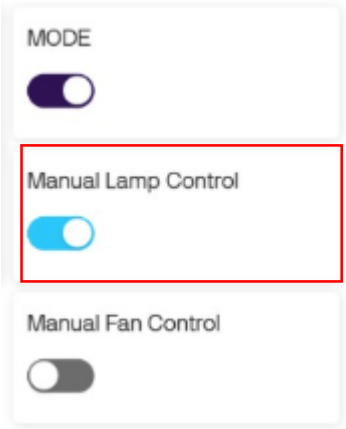


Figure 6. 6: Dual-Mode Control for Bulb (Heating Element)

CHAPTER 7: PROBLEMS AND SOLUTIONS

A number of difficulties developed during the development and integration of the control and communication components of the IoT Wireless Weather Monitoring System. These problems mostly happened when setting up dual-mode operation, virtual pin assignments, cloud data storage, and Wi-Fi and Blynk connectivity. To guarantee that the system operated dependably, each issue needed to be approached methodically in order to determine its underlying cause and put workable solutions in place. The issues encountered, their root causes, and the related fixes are listed in **Table 7.1** below.

Table 7. 1:Problems and Solutions

Problems	**Causes**	**Solutions**
ESP32 not connecting to Wi-Fi or Blynk Cloud	Incorrect Wi-Fi credentials or unstable 2.4GHz signal	Verified and corrected Wi-Fi details; ensured strong signal; restarted ESP32
Virtual pins not responding on Blynk dashboard	Incorrect virtual pin mapping or missing Blynk.virtualWrite() statements	Checked virtual pin assignments; corrected code; ensured widgets linked to correct pins
Manual mode switch not controlling fan and bulb	Missing BLYNK_WRITE () functions for manual control	Added BLYNK_WRITE(V4) and BLYNK_WRITE(V5) to read switch states and control relays accordingly
Dual-mode logic conflict between Automatic and Manual modes	Mode control logic was overlapping or poorly isolated	Separated control logic using if (mode == 0) for automatic and else for manual
Inconsistent or unrealistic sensor readings displayed	Sensor disconnected or values read too frequently	Stabilized sensor wiring; added delays between readings; validated values before sending to Blynk
Cloud History Data not recording sensor values	History Data feature not enabled for relevant data streams	Enabled **History Data** for V0 (Temp), V1 (Humidity), and V2 (Light) in Blynk dashboard

CHAPTER 8: TECHNICAL LESSONS LEARNT

- Processes that Worked Well

1. Breaking down complex tasks: Dividing the project into smaller tasks, such as researching communication methods and designing the GUI, helped to manage complexity and make progress.
2. Consulting documentation and resources: Referring to official documentation, tutorials, and online resources helped to resolve technical issues and gain new knowledge.

- Problem-Solving Strategies:

1. Identifying potential problems: Anticipating potential difficulties and having corrective actions in place helped to mitigate risks and ensure project progress.
2. Testing and debugging: Thoroughly testing and debugging the system helped to identify and resolve technical issues.

- Hints for other Project Manager:

1. Stay organized: Keep track of progress, notes, and resources to ensure efficient project management.
2. Be proactive: Anticipate potential problems and take corrective action to mitigate risks.

- Unexpected Outcomes:

1. Gaining new skills and knowledge: Working on this project has provided valuable experience with Blynk, ESP32, and GUI design.
2. Improved problem-solving skills: The project has helped develop problem-solving strategies and critical thinking.

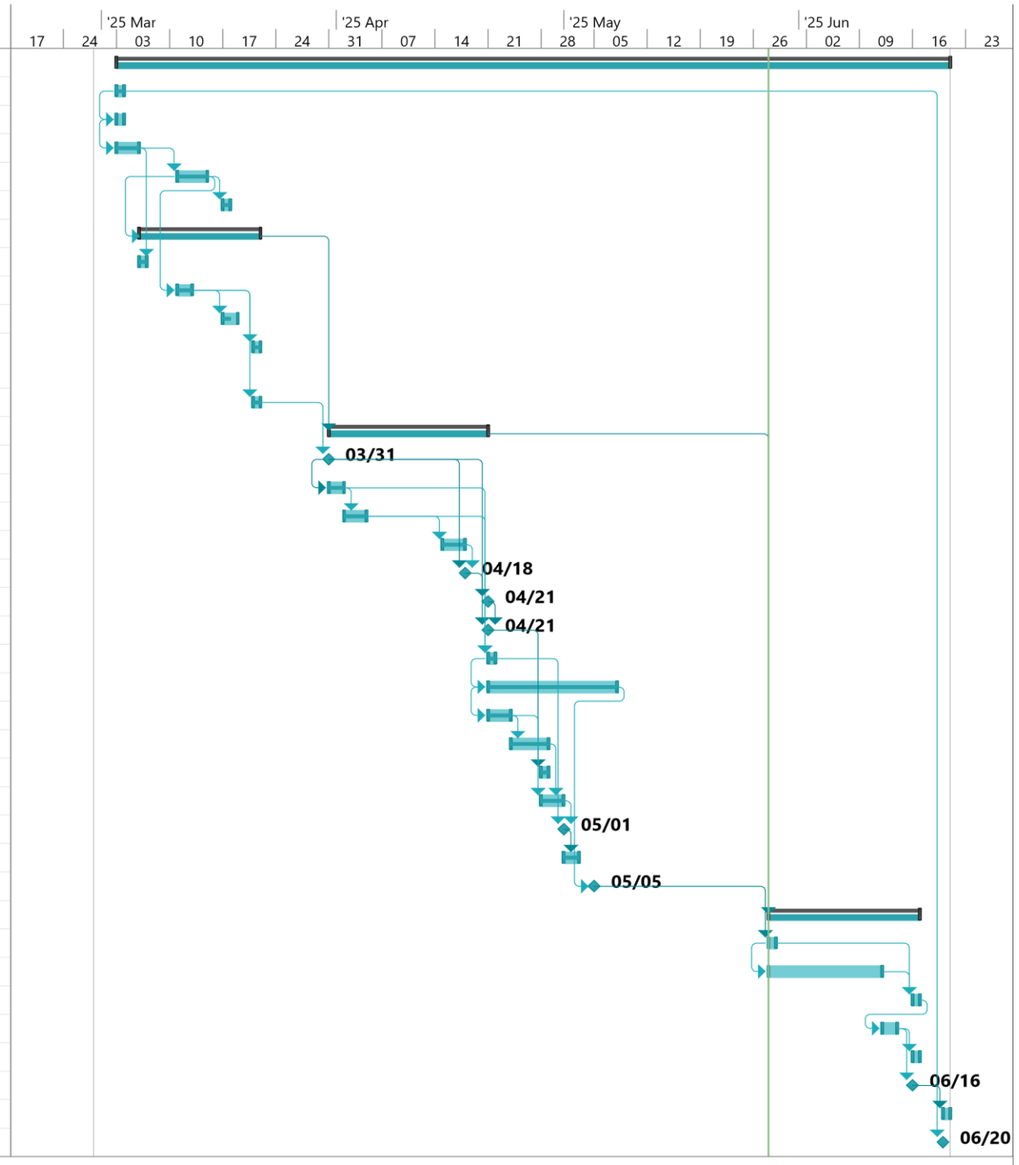
- Added Value:

1. Practical experience: The project has provided hands-on experience with IoT development and automation systems.
2. Applicability to future projects: The skills and knowledge gained can be applied to future projects, enhancing their success.

CHAPTER 9: TIME MANAGEMENT

Figure 9.1 shows the Gannt chart that was used in individual contribution in the group to easily communicate the project plan with the team, avoid resource overload, track project progress, complete projects ahead of schedule, and increase productivity. Development of the model was aided by the software learnt in.

ID	Task Mode	Task Name	Duration	Start	Finish	Predecessors
1		**COMMUNICATION SYSTEM**	**80 days**	**Mon 25/03/03**	**Fri 25/06/20**	
2	✓	Start Project	1 day	Mon 25/03/03	Mon 25/03/03	
3		Group meetings	1 day	Mon 25/03/03	Mon 25/03/03	2SS
4	✓	Topic allocation(IoT Engineer)	3 days	Mon 25/03/03	Wed 25/03/05	3SS
5	✓	Communication research	4 days	Tue 25/03/11	Fri 25/03/14	4
6	✓	Component Search	1 day	Mon 25/03/17	Mon 25/03/17	5
7		**Final proposal**	**12 days**	**Thu 25/03/06**	**Fri 25/03/21**	**5SS**
8	✓	Task planning	1 day	Thu 25/03/06	Thu 25/03/06	4
9	✓	Research communication method	2 days	Tue 25/03/11	Wed 25/03/12	5
10		Drawing circuit diagram	2 days	Mon 25/03/17	Tue 25/03/18	9
11	✓	Combining communication proposal information	1 day	Fri 25/03/21	Fri 25/03/21	9
12	✓	Group submission of the task	1 day	Fri 25/03/21	Fri 25/03/21	9
13	✓	**Project Progress Report**	**15 days**	**Mon 25/03/31**	**Mon 25/04/21**	**7**
14	✓	Start Progress report	0 days	Mon 25/03/31	Mon 25/03/31	12
15	✓	Step-up BLYNK account	2 days	Mon 25/03/31	Tue 25/04/01	14SS
16	✓	Set-up Blynk widget	3 days	Wed 25/04/02	Fri 25/04/04	15
17	✓	Start progress presentation slide	3 days	Tue 25/04/15	Thu 25/04/17	16
18	✓	Progress presentation Submition	0 days	Fri 25/04/18	Fri 25/04/18	14,17
19	✓	Presentation	0 days	Mon 25/04/21	Mon 25/04/21	18
20	✓	Finish progress	0 days	Mon 25/04/21	Mon 25/04/21	14,19
21	✓	Start wireless communication	1 day	Mon 25/04/21	Mon 25/04/21	15,16
22	✓	Wirelss communication	13 days	Mon 25/04/21	Wed 25/05/07	21SS
23	✓	Code ESP32 for Wi-Fi connection	3 days	Mon 25/04/21	Wed 25/04/23	22SS
24	✓	Code ESP32 to send Data to BLYNK	3 days	Thu 25/04/24	Mon 25/04/28	23
25	✓	Individual Progress Report Submission	1 day	Mon 25/04/28	Mon 25/04/28	20
26	✓	Take results	3 days	Mon 25/04/28	Wed 25/04/30	23,24
27	✓	Finish Wi-Fi communication	0 days	Thu 25/05/01	Thu 25/05/01	21,26
28	✓	Cloud Storage implementation	2 days	Thu 25/05/01	Fri 25/05/02	27
29	✓	Finish wirelss communication	0 days	Mon 25/05/05	Mon 25/05/05	22
30		**Final report**	**14 days**	**Wed 25/05/28**	**Mon 25/06/16**	**13**
31		Start final individual report	1 day	Wed 25/05/28	Wed 25/05/28	29
32		Research individual report	11 days	Wed 25/05/28	Wed 25/06/11	31SS
33		Final report submission	1 day	Mon 25/06/16	Mon 25/06/16	31,32
34		Presentation slide	2 days	Thu 25/06/12	Fri 25/06/13	33
35		Presentation slides submission	1 day	Mon 25/06/16	Mon 25/06/16	34
36		Presentation preparation	0 days	Mon 25/06/16	Mon 25/06/16	34
37		Project presentation	1 day	Fri 25/06/20	Fri 25/06/20	36
38		Finish project	0 days	Fri 25/06/20	Fri 25/06/20	2



Project: CHAPTER 4...
Date: Wed 25/05/28

Task	Project Summary	Manual Task	Start-only	Deadline	
Split	Inactive Task	Duration-only	Finish-only	Progress	
Milestone	Inactive Milestone	Manual Summary Rollup	External Tasks	Manual Progress	
Summary	Inactive Summary	Manual Summary	External Milestone		

Figure 9. 1: Gantt Chart

CHAPTER 10: CONCLUSION

In this report a detailed information of the individual contribution to IoT wireless weather monitoring system is provided, this individual task focused on designing and implementing a user-friendly GUI, developing wireless communication, and setting up a Blynk account for the greenhouse automation system. Using the Blynk IoT platform to provide wireless communication, remote monitoring, user interface design, cloud storage, and dual-mode control was the main focus of the individual contribution to the IoT Wireless Weather Monitoring System project. Steps to make sure that real-time data transfer and remote accessibility of sensor values was taken, such as temperature, humidity, and light intensity, were possible by successfully establishing Wi-Fi connectivity between the ESP32 and Blynk.

With the use of virtual pins, the Blynk GUI was successfully created to show real-time environmental data and offer simple manual control over the fan and lightbulb, which are the actuators of the system. Utilising History Data feature of Blynk to integrate cloud storage allowed for the historical examination of environmental trends, which enhanced the value of the system. Additionally, by including a dual-mode operation, the system was able to alternate between manual and automatic management, offering users flexibility and adaptability.

Overall, the contributions improved the intelligence of the system, usability, and usefulness, bringing it into line with contemporary IoT-based smart agriculture solutions. The knowledge of wireless communication, real-time data handling, and cloud-based IoT system design has also increased as a result of the experience.

REFERENCES

[1] A. Badji, A. Benseddik, H. Bensaha, A. Boukhelifa, and I. Hasrane, "Design, technology, and management of greenhouse: A review," *Journal of Cleaner Production,* vol. 373, p. 133753, 2022.

[2] S. S. Samsuri, M. Yusof, and M. A. Othman, "Development of Internet of Things (IoT) Based by Using Blynk for Irrigation and Fertigation System," *Politeknik & Kolej Komuniti Journal of Engineering and Technology,* vol. 9, no. 2, pp. 108–124, 2024.

[3] M. Artiyasa, I. H. Kusumah, A. Suryana, A. D. W. M. Sidik, and A. P. Junfithrana, "Comparative Study of Internet of Things (IoT) Platform for Smart Home Lighting Control Using NodeMCU with Thingspeak and Blynk Web Applications," *FIDELITY: Jurnal Teknik Elektro,* vol. 2, no. 1, pp. 1–6, 2020.

[4] S. Ogunbunmi, A. Taiwo, J. B. Oladosu, H. Sanusi, F. Inaolaji, and U. Olasunkanmi, "Internet of things weather monitoring system," *World Journal of Advanced Research and Reviews,* vol. 22, no. 2, pp. 2099–2110, 2024.

[5] A. Costantino, E. Fabrizio, and S. Calvet, "The role of climate control in monogastric animal farming: The effects on animal welfare, air emissions, productivity, health, and energy use," *Applied Sciences,* vol. 11, no. 20, p. 9549, 2021.

[6] A. Rivera, P. Ponce, O. Mata, A. Molina, and A. Meier, "Local weather station design and development for cost-effective environmental monitoring and real-time data sharing," *Sensors,* vol. 23, no. 22, p. 9060, 2023.

[7] P. Susmitha and G. S. Bala, "Design and implementation of weather monitoring and controlling system," *International journal of Computer applications,* vol. 97, no. 3, 2014.

[8] A. P. Selvam and S. N. S. Al-Humairi, "Environmental impact evaluation using smart real-time weather monitoring systems: a systematic review," *Innovative Infrastructure Solutions,* vol. 10, no. 1, pp. 1–24, 2025.

[9] Y. Rahut, R. Afreen, D. Kamini, and S. S. Gnanamalar, "Smart weather monitoring and real time alert system using IoT," *International Research Journal of Engineering and Technology,* vol. 5, no. 10, pp. 848–854, 2018.

[10] P. B. Leelavinodhan, M. Vecchio, F. Antonelli, A. Maestrini, and D. Brunelli, "Design and implementation of an energy-efficient weather station for wind data collection," *Sensors,* vol. 21, no. 11, p. 3831, 2021.

[11] Y.-C. Tsao, Y. Te Tsai, Y.-W. Kuo, and C. Hwang, "An implementation of iot-based weather monitoring system," in *2019 IEEE International Conferences on Ubiquitous Computing & Communications (IUCC) and Data Science and Computational Intelligence (DSCI) and Smart Computing, Networking and Services (SmartCNS)*, 2019: IEEE, pp. 648–652.

[12] W. O. Galitz, *The essential guide to user interface design: an introduction to GUI design principles and techniques*. John Wiley & Sons, 2007.